

ByteNet: Rethinking Multimedia File Fragment Classification Through Visual Perspectives

A Reproduction Study – Implementation, Training & Results

Payaldurga Paila Mehul Arpita

Indian Institute of Information Technology Vadodara
Under the guidance of Prof. Jignesh Patel

Problem Definition

A fragment is a fixed-size chunk of a file – 512 bytes off a disk sector, say, or out of a network packet – with no filename, no extension, and nothing attached to say what kind of file it came from.

Guessing the file type from that chunk alone is what file fragment classification tries to do, and it's harder than it sounds:

- a fragment can come from anywhere in a file, not just the start, so a magic-byte signature like FFD8 for JPEG isn't guaranteed to even be present
- most prior learning-based methods – Byte2Vec, FiFTy, DSCNN – read a fragment as a flat 1D sequence, so they only ever compare a byte to *other separate bytes* (interbyte relationships)

In practice this comes up in digital forensics, pulling files off damaged storage, and in network security, inspecting traffic where headers have been stripped or corrupted.

Why Interbyte-Only Methods Fall Short

Byte2Image is built around a specific gap in the sequence-only view: a byte has 8 bits, and nothing forces the patterns worth learning to line up neatly with byte boundaries.

JPEG is the clean example. Its DCT coefficients are Huffman coded, and Huffman codes are variable-length – a single symbol (the paper uses the pair “(4)” and “(-9)” in its own Fig. 3) can start in the middle of one byte and finish in the middle of the next. A model that only ever compares byte N to byte $N+1$ has no way to see that pattern, because the pattern doesn't respect byte boundaries at all.

The fix is almost mechanical: bit-shift the fragment seven times and stack the results, and the bits inside a byte become just as visible to the model as the bytes themselves.

Byte2Image: The Transform

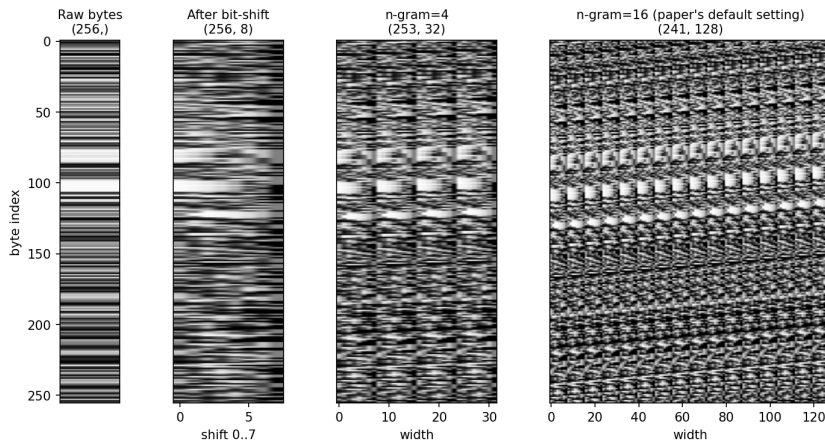
Turning a byte fragment into an image happens in three steps, each one addressing a problem the step before it leaves behind:

- 1 **Bit-shift.** Given a fragment of N_s bytes, compute $x_0 = x$ and $x_i = (x_{i-1} \ll 1) \bmod 256$ for $i = 1, \dots, 7$. Stacking these seven shifted copies row-wise gives a matrix $x_{\text{in}} \in \mathbb{Z}^{N_s \times 8}$: each *row* carries intrabyte (bit-level) structure, each *column* keeps the original interbyte structure intact.
- 2 **Intrabyte n-gram.** x_{in} is only 8 pixels wide, which convolutional layers handle badly. Grouping n consecutive rows and concatenating them column-wise produces a near-square image of shape $(N_s - n + 1, 8n)$ instead.
- 3 **Augment.** Horizontal flip, random erase, and mixup – standard moves, needed here because the fragment count itself can't grow.

Byte2Image in Action – Real Output

Nothing here is a mockup – this came straight out of running our own `bytenet/byte2image.py` on a real 256-byte PNG fragment pulled from the dataset:

Byte2Image transform running on a real 256-byte PNG fragment from our dataset



Code: The Bit-Shift Step

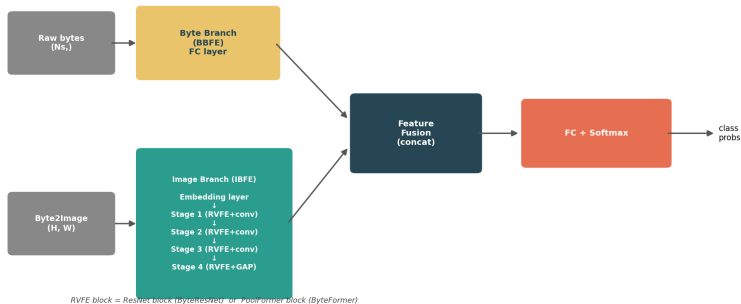
This is the core of Byte2Image as it sits in bytenet/byte2image.py:

```
def bit_shift_stack(byte_seq: np.ndarray) -> np.ndarray:
    """Eq. (2):  $x_0 = x$ ,  $x_i = (x_{i-1} \ll 1) \bmod 256$ ."""
    byte_seq = np.asarray(byte_seq, dtype=np.uint8)
    ns = byte_seq.shape[0]
    x_in = np.empty((ns, 8), dtype=np.uint8)
    cur = byte_seq.copy()
    for i in range(8):
        x_in[:, i] = cur
        cur = ((cur.astype(np.uint16) << 1) & 0xFF).astype(np.uint8)
    return x_in
```

It's a near-literal reading of the paper's Eq. (2) – each output column is the fragment shifted one bit further left than the column before it.

ByteNet Architecture

ByteNet dual-branch architecture (as implemented in bytenet/models.py)



The byte branch is deliberately small – one fully-connected layer over the raw bytes, mainly there to catch magic-byte co-occurrences like FFD8 for JPEG. The image branch does the heavy lifting: an embedding layer feeds four stages of residual blocks, each one downsampling before the next, ending in a global average pool. Concatenate what comes out of both branches, run it through one more linear layer with softmax, and that's the whole classifier.

Code: Dual-Branch Fusion

And here's that fusion step for real, straight out of the `forward()` method of our ByteNet class in `bytenet/models.py`:

```
def forward(self, raw_bytes, img):
    # raw_bytes: (B, Ns) in [0,1]; img: (B, 1, H, W) in [0,1]
    x_sf = self.byte_branch(raw_bytes)

    x = self.embed(img)
    for stage in self.stages:
        x = stage(x)
    x_df = x.flatten(1)  # after GAP: (B, C4, 1, 1) -> (B, C4)

    fused = torch.cat([x_sf, x_df], dim=1)
    logits = self.classifier(fused)
    return logits
```

`x_sf` is what the byte branch produces, `x_df` is the image branch's output after its last global average pool – concatenating them is literally the fusion box in the diagram, nothing more exotic.

Two Variants: ByteResNet vs. ByteFormer

The image branch can be built from two different block types, which is where the paper's two named variants come from – both live in our RVFESStage class:

ByteResNet

- wide n-gram embedding convolution
- classic ResNetBlock: two 3×3 convs, batch norm, residual connection
- 489K parameters

ByteFormer

- patch embedding, splits the image into patches
- PoolFormerBlock: a pooling-based token mixer plus a channel MLP, no attention involved
- 186K parameters, and roughly $15\times$ faster per epoch on CPU

We trained both, end to end, to see how they actually compare rather than take the paper's word for it.

Building a Real Dataset

FFT-75 and VFF-16, the benchmarks the paper actually trains on, weren't reachable from where we were building this. Rather than feed the network random noise and call it a test, we pointed `bytenet/data_prep.py` at the local filesystem instead: seven file types, and for each one, 256-byte windows cut from random offsets inside real files – `.py`, `.json`, `.html`, `.png`, `.pdf`, `.gz`, and ELF binaries. That's close to how FFT-75 itself was assembled, just from GovDocs documents instead of whatever happened to be sitting on this machine.

- 320 fragments per class, 2,240 total
- 80/20 train/test split

Because the offsets are random, a fragment landing on a file's header is mostly luck – the model can't lean on magic bytes being there.

Dataset Build – Real Console Output

What actually printed when we ran the builder:

```
$ python -m bytenet.data_prep --sector-size 256 --per-class 320
Saved 2240 sectors of size 256 bytes -> data/fragments.npz
Classes: ['elf', 'gz', 'html', 'json', 'pdf', 'png', 'py']
elf      : 320 sectors from 2637 files
gz       : 320 sectors from 1376 files
html     : 320 sectors from 398 files
json     : 320 sectors from 1470 files
pdf      : 320 sectors from 804 files
png      : 320 sectors from 551 files
py       : 320 sectors from 4000 files
```

Every class draws its 320 fragments from hundreds to thousands of distinct source files – no single file is carrying an entire class.

Training Setup

We stuck to the paper's training recipe (Sec. IV-B) as closely as a single CPU core allows:

Setting	Value
Optimizer	AdamW, weight decay 0.01
LR schedule	Cosine decay, linear warmup
Loss	Negative log-likelihood on mixup-soft labels
Batch size	64
Augmentation	Horizontal flip, random erase, mixup
Stage channels	[16, 32, 64, 128]
Epochs	16 (ByteResNet) / 60 (ByteFormer)
Compute	1 CPU core

Training Log – Real Console Output

ByteResNet, full dual-branch, n-gram=4 – a few epochs out of the full run:

```
[resnet/full/n4] epoch 1/16 loss=1.7041 train_acc=0.236
test_acc=0.125
[resnet/full/n4] epoch 4/16 loss=1.3688 train_acc=0.451
test_acc=0.496
[resnet/full/n4] epoch 8/16 loss=1.1675 train_acc=0.408
test_acc=0.551
[resnet/full/n4] epoch 10/16 loss=1.2040 train_acc=0.493
test_acc=0.696
[resnet/full/n4] epoch 13/16 loss=0.9911 train_acc=0.448
test_acc=0.723
[resnet/full/n4] epoch 16/16 loss=1.0619 train_acc=0.488
test_acc=0.734
Saved results to outputs/resnet_full
```

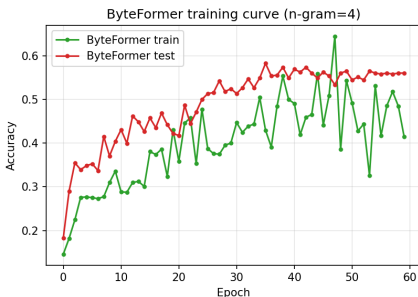
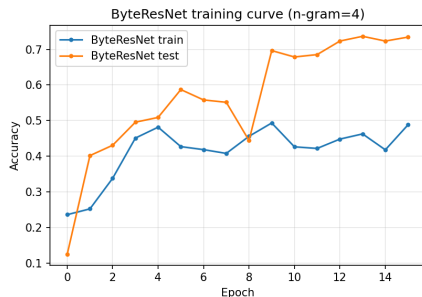
1,792 fragments for training, 448 held out for testing. Final number:
73.4% test accuracy off 489,695 trainable parameters.

Results Summary

Configuration	Test Acc.	Params
ByteResNet (full)	73.4%	489K
ByteFormer (full)	56.0%	186K
ByteResNet – byte branch only	30.8%	489K
ByteResNet – image branch only	75.4%	489K
ByteResNet, n-gram = 2	73.2%	488K
ByteResNet, n-gram = 8	72.5%	490K
ByteResNet, n-gram = 16	72.1%	492K

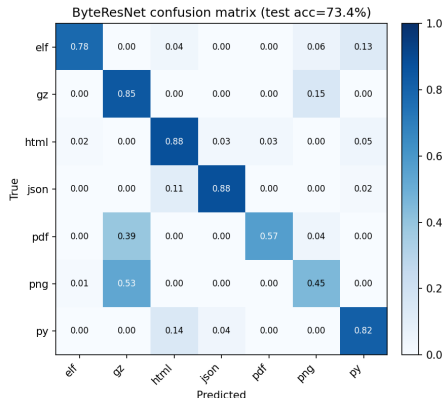
- Guessing at random across 7 classes gets you 14.3%.
- Full ByteResNet clears five times that.
- Most of the gap comes from the image branch, not the byte branch – more on that in a few slides.

Training Curves



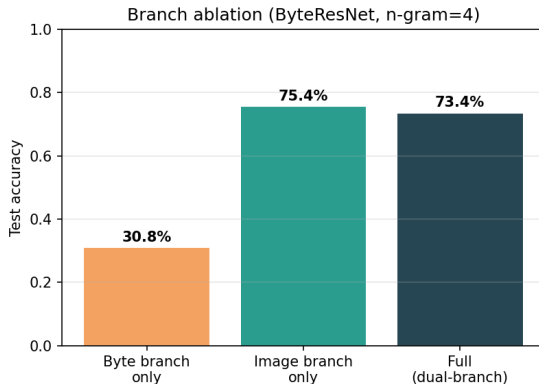
Train accuracy sits below test accuracy for a mundane reason: it's computed on mixup-blended batches against soft labels, so it's noisier by construction, not because the model is struggling. ByteFormer, on the right, needed almost four times the epochs just to get into the same neighborhood, and still didn't fully catch up.

Confusion Matrix – Where Errors Happen



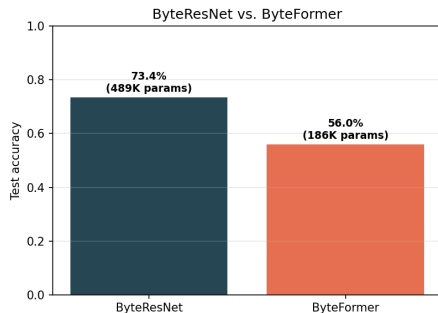
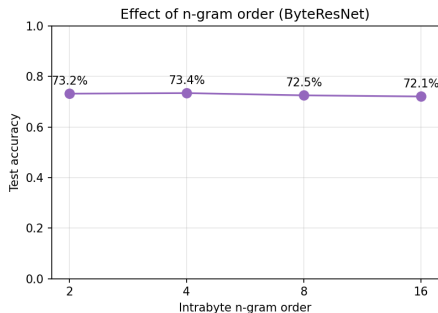
pdf, png, and gz keep getting mixed up with each other, and there's a real reason for it: PDF embeds zlib-compressed streams internally, and PNG uses zlib for its own compression – at the byte level, compressed data from different containers just looks similar. The original paper runs into the same wall between its Archive and Published categories (their Fig. 7).

Branch Ablation



Take the image branch away and accuracy drops to 30.8% – barely above guessing. Take the byte branch away instead and it actually goes up slightly, to 75.4%. It's the same pattern the paper reports in its Table V: the image branch carries almost the entire result, and the byte branch adds a smaller signal on top of it.

N-gram Sweep & Variant Comparison



On the left: moving the n-gram order from 2 up to 16 only shifts accuracy by about a point, which lines up with the paper calling ByteResNet fairly insensitive to this setting.

On the right: ByteFormer trains roughly $15\times$ faster per epoch, but even at 60 epochs against ByteResNet's 16, it settles well below it.

Honest Discussion: Where We Diverge

A few places our numbers don't line up with the paper, laid out plainly rather than buried:

- the image-only ablation (75.4%) edges out the full model (73.4%) – in the paper, full always wins. Our best guess is that the byte branch needs FFT-75-scale data, on the order of 100K fragments per class, before it actually pulls its weight. We're running 320.
- ByteFormer falls further behind here than it does in the paper, even with nearly four times the epoch budget – patch-based tokenization probably just needs more data than 2,240 fragments can give it.
- none of these numbers are directly comparable to FFT-75 or VFF-16 results anyway – this is a different, much smaller dataset.

What ties all three together is scale, not a bug in the implementation – everything we could actually test at this size came out the way the paper says it should.

Our Contribution – What We Implemented

None of this is adapted from the authors' repository. Everything below is code we wrote and ran ourselves:

- `bytenet/byte2image.py` – the bit-shift and n-gram transform, unit-tested on its own before it touched the rest of the pipeline
- `bytenet/models.py` – the byte branch, both embedding layers (n-gram and patch), both block types (ResNet and PoolFormer), the four-stage backbone, and the fusion step
- `bytenet/data_prep.py` – the real 7-class dataset builder
- `bytenet/train.py` – the AdamW plus cosine schedule plus mixup training loop, with byte-only and image-only ablation modes built in
- `scripts/` – the experiment runner and figure generator that produced every chart in this talk

Thank You

Code: `github.com/<your-username>/bytenet-implementation`

Every figure, log, and number in this talk came out of that repository –
nothing here is mocked up.

Questions?